

BABA IS YOU

Badr MOTAIM
Yendoumban NANMEDIGOU

Rapport de Projet

Janvier 2026
Université de Paris Cité

Contents

1	Introduction	2
2	Fonctionnalités	2
3	Architecture	2
3.1	Utils	3
3.2	Characters	3
3.3	Core	3
3.4	Loader	3
3.5	View	4
4	Moteur de jeu	4
4.1	Gestion des déplacements	4
4.2	Undo/Redo	5
4.3	Gestion des règles	5
5	Gestion de mémoire	5
6	Les plus réalisés	5
7	Notes	6
8	Conclusion	6

1 Introduction

“Baba Is You” est un jeu de réflexion qui se représente sur une grille bidimensionnelle dans laquelle un personnage se déplace pour atteindre un objectif. Il devra pour cela interagir avec les objets présents. Notre projet consiste en une implémentation de ce jeu en langage C++, avec une interface graphique basée sur la bibliothèque SFML. L’objectif principal a été de concevoir une architecture claire et évolutive permettant la gestion dynamique des règles du jeu, tout en respectant les principes de gestion de mémoire (construction, copie, destruction...) et de conception abordés en cours (héritage, design patterns...). Un focus sur ces points a eu pour coût la limitation de fonctionnalités par rapport au jeu original qui comporte en réalité une multitudes de règles et de personnages.

2 Fonctionnalités

Le point d’entrée de notre jeu est notre interface graphique soignée qui propose deux modes de jeu ; le mode normal où l’accès aux niveaux supérieurs est conditionné par le déblocage des niveaux antérieurs. Puis le mode explorer qui nous permet d’accéder inconditionnellement aux niveaux, pensé pour faciliter les démos et les phases de tests. Quand on joue, l’on a accès à un environnement où les règles fondamentales de BABA IS YOU sont codées. Pour une meilleure jouabilité, les règles sont dynamiques ; c’est à dire qu’on peut créer une règle, la détruire et la recréer à guise et les changements seront pris en compte. Cette même jouabilité a été le motif de l’implémentation des fonctionnalités **undo** (Ctrl Z) et **redo** (Ctrl Y). Un total de 11 niveaux ont été implémentés (tous les 8 du site kbhgames-babaisyou et 3 niveaux originaux) et l’on peut rajouter facilement ses propres niveaux. Somme toute, le travail fourni offre un jeu BABA IS YOU complet, exception faite des personnages non traités.

3 Architecture

L’architecture du projet repose sur une séparation claire entre la logique du jeu et son affichage. Le modèle central s’articule autour de la classe **Game**, qui fait interagir entre eux des **Character** sur un **Board** tout en restant attentif aux notifications de son **Ruler**. Les détails sont dans les lignes à suivre et partent des utilitaires utilisés pour aller vers l’élément **Game** central.

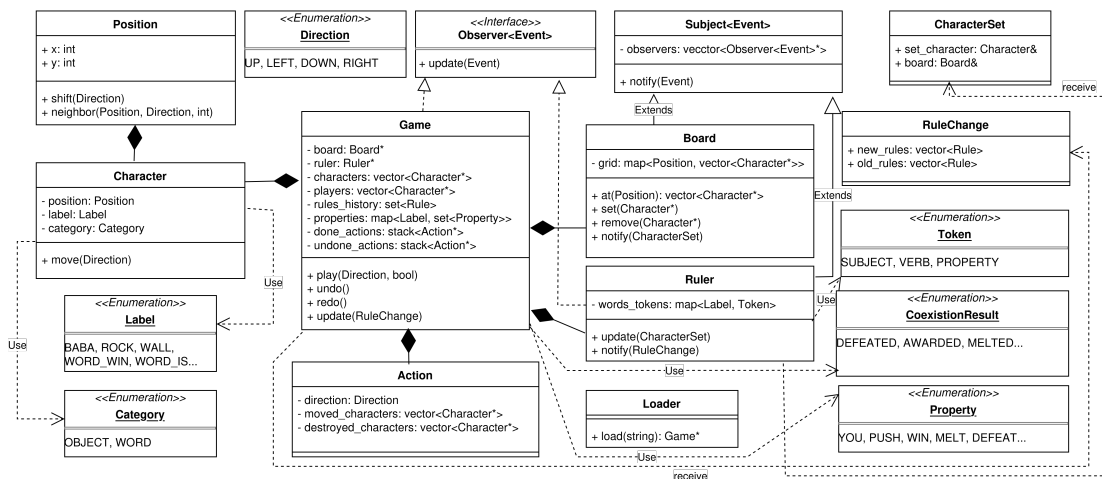


Figure 1: Diagramme UML

Live Rule = tuple<Label, Label, Label>

3.1 Utils

Observer Pattern : La classe `Subject<Event>` et l'interface `Observer<Event>` nous permettent d'implémenter le design pattern Observer. Le template `Event` nous permet de décider de la nature des faits observés et des paramètres à transmettre lors des notifications. Dans notre cas, `Ruler` écoute `Board` qui le notifie des `CharacterSet` events. À son tour, il notifie `Game` des `RuleChange` events qu'il aurait détecté après traitement des `CharacterSet` events reçus.

Position : La classe `Position` nous permet d'encapsuler les positions de nos `Character`. On a la possibilité de les shifter dans une direction donnée ou d'en récupérer le voisin dans une direction et un périmètre donnés. Ces traitements faits par `Position` permettront à terme d'alléger ceux de `Game`.

3.2 Characters

Un `Character`, personnage de notre jeu donc, a une position, un label (`BABA`, `ROCK`, `WORD_IS...`) et une category (`WORD` ou `OBJECT`). Il peut se déplacer dans une direction donnée, ce qui est la seule logique du jeu qu'il renferme. La responsabilité de ses propriétés a été déléguée à `Game` ; d'un côté car les règles les concernant sont dynamiques ; et de l'autre, nous n'avons pas trouvé judicieux de dupliquer l'information des propriétés d'un `WALL` par exemple dans les n `WALL` potentiellement présents sur le plateau. Nous avons choisi des labels et non des sous classes pour différencier le comportement des `Character` en fonction de leurs propriétés une fois de plus à cause du caractère dynamique des règles. Si un objet se comporte d'une certaine manière en raison d'une propriété X qu'il possède, on ne pourra modifier la nature (le type) de cet objet lorsqu'il n'aura plus la propriété X . Une solution serait peut être détruire cet objet et de le remplacer par un autre qui respecte la nouvelle règle créée. Mais conférer juste des labels aux `Character` et laisser `Game` décider de leur propriétés en fonction de l'état de ses règles est beaucoup plus simple et moins coûteux.

3.3 Core

Board : C'est notre plateau de jeu. Il enregistre à des positions données le ou les `Character` qui s'y trouvent. Nous avons choisi une `map<Position, vector<Character*>>` pour la grid plutôt qu'une matrice (`vector<vector<vector<Character*>>>`) car souvent sur un plateau, beaucoup de positions sont inoccupées. La vue devra rendre compte de cela, mais `Board` dans notre modèle n'a pas besoin de stocker ces positions. Cela réduit le coût en espace. Lorsque `Game` dépose ou retire un `Character` du plateau, il indique à `Board` si ce dernier doit notifier `Ruler`. Si oui, cette notification sera encapsulée dans un `CharacterSet` event qui contient une référence vers lui-même le `Board` et le `Character` déposé ou retiré.

Ruler : C'est le détecteur de changement de règles. Plutôt que de rescanner tout le board à chaque tour de jeu et faire la différence avec les règles présentes au tour $t-1$, `Ruler`, lorsque seulement un mot est déplacé scanne tous les triplets de positions à proximité de cet mot qui pourraient signaler une création ou destruction de règle. Il y'en a 6 pour les créations et 12 pour les destructions de règle contre un nombre de l'ordre $width*height$ si on devait scanner tout le plateau. Le prix de cette efficacité est la possibilité de détection de fausses anciennes règles. Mais `Game` (récepteur des notifications de `RuleChange` event qu'émet `Ruler`) garde l'historique des règles et saura donc s'en apercevoir.

Game : C'est le coeur du modèle. Son fonctionnement sera explicité dans la partie **Moteur de jeu**.

3.4 Loader

`Loader` nous permet de charger un `Game` à partir d'un fichier de config `.txt`. Cela nous évite de coder en dur les différents niveaux et offre une extensibilité (création de niveaux très facile). Le format est très simple. On commence par mentionner les dimensions du plateau (24 14 par exemple),

ensuite les characters en indiquant leur position, leur label et leur catégorie (2 11 ROCK 0 par exemple), puis les règles (WORD_BABA WORD_IS WORD_YOU par exemple). Et c'est tout ! Il faudra juste faire attention à déclarer les règles de la forme SUBJECT IS SUBJECT (WALL IS BABA par exemple) après celles concernant les deux sujets impliqués. Les objets sont en effet créés au fur et à mesure de la lecture du fichier pour une meilleure efficacité.

3.5 View

La vue (en réalité vue/contrôleur) de notre jeu est représentée par la classe `BoardView` qui disposant d'une référence vers `Game` et `Board` lit les entrées de l'utilisateur et modifie `Game` en conséquence.

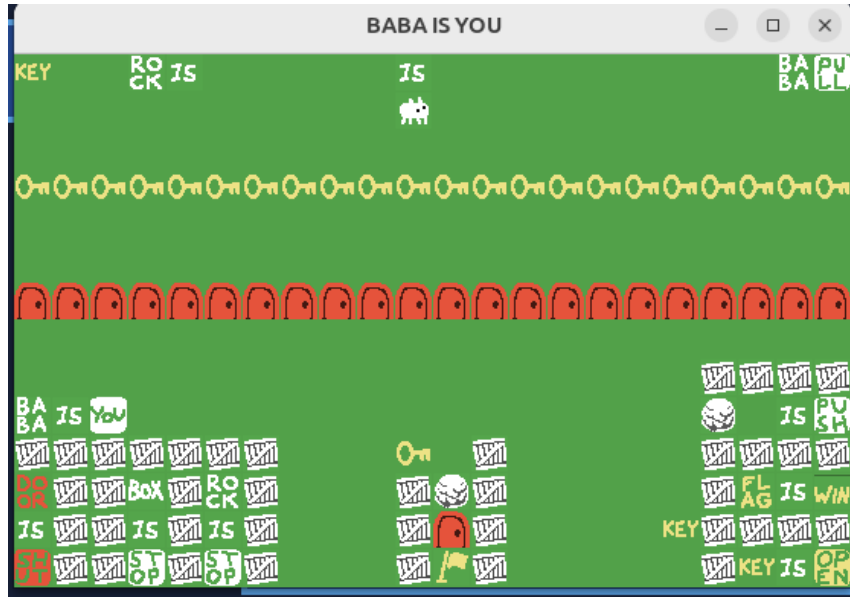


Figure 2: État initial du niveau 10 original de notre jeu

4 Moteur de jeu

4.1 Gestion des déplacements

La fonctionnalité de jeu réside en la méthode `play(Direction, bool)` de la classe `Game`. On veut aller dans la direction indiquée et l'on veut ou pas tirer à notre suite les objets derrière nous dans la mesure du possible. `Game` a une liste de `players` (characters qui ont la propriété `YOU`) à qui il applique l'intention de move l'un après l'autre.

La phase de déplacement se déroule en trois étapes. D'abord une chaîne de push est calculée dans la direction du déplacement ; -1 si le move est interdit en raison d'un objet bloquant ou d'une limite du plateau, 0 si seul le player bougera et n si n objets bougeront devant lui comme résultat de son action. Dans ce dernier cas, l'objet à la fin de la chaîne est d'abord déplacé, puis l'avant dernier, ainsi de suite... Ensuite, le player est déplacé. Enfin une chaîne de pull est calculée (si le joueur en a indiqué l'intention bien sûr) et les objets concernés sont déplacés, d'abord le premier élément de la chaîne cette fois ci. Les trois phases fusionnées donnent ainsi une chaîne de déplacements ordonnée du premier objet concerné au dernier.

Lorsqu'un `Character` (qu'il soit player ou pas) se déplace dans une case voisine, ses propriétés interagissent avec celles de chacun de ses voisins pour donner un `CoexistenceResult`. Les propriétés `YOU + DEFEAT` donneront `DEFEATED` par exemple, `MELT + HOT = MELTED`, n'importe quelle propriété + case vide = `COEXISTED` (résultat par défaut)... Des éventuels nombreux `CoexistenceResult` calculés pour chacun des voisins, un seul prioritaire est choisi et appliqué. Ce choix découle souvent de la lecture des règles fondamentales du jeu (`MELTED` est prioritaire à `COEXISTED`), mais

est aussi parfois arbitraire de notre part (**DEFEATED** est prioritaire par rapport à **AWARDED**). Ce **CoexistenceResult** décidera donc du prochain état de **Game**: terminé sur une défaite ou une victoire, certains characters défaits et supprimés, d'autres déplacés...

4.2 Undo/Redo

Comment revenir à un état antérieur du jeu, ou même depuis un état antérieur repartir vers ceux déjà rencontrés ? Faut-il garder à chaque action de l'utilisateur une copie de **Game** et déplacer ensuite un curseur à guise pour implémenter le undo/redo ? Cette solution a été écartée car moins modulaire et plus coûteuse (des copies à prévoir). Nous avons donc décidé d'encapsuler dans des **Action**, pour chaque tour de jeu, toutes les modifications de **Game**. Cela consiste en une **Direction** de déplacement, un vecteur de **Character*** déplacés et un autre pour ceux détruits ; ce qui est plutôt léger. Défaire une action sera déplacer les characters déplacés par l'action dans la direction inverse et replacer sur le plateau ceux détruits par l'action. Refaire une action sera déplacer les characters déplacés par l'action dans la même direction et retirer du plateau ceux détruits par l'action. Les deux piles **done_actions** et **undone_actions** dans **Game** servent à cela.

4.3 Gestion des règles

Les règles du jeu sont codées dans **Game** par sa **map<Label, set<Property>> properties** qui à un instant *t* dit qui a quelle propriété. C'est en consultant cette map qu'elle arrive à résoudre les interactions de ses characters lors des déplacements. Le dynamisme de ces règles est assurée par les **RuleChange** event qu'il reçoit de **Ruler**. Un **RuleChange** event encapsule une liste de nouvelles règles et une liste d'anciennes règles (**vector<Rule>**). On entend par règle ici un tuple de trois **Label** (**WORD_ROCK**, **WORD_IS**, **WORD_WIN** par exemple). **Game** applique les nouvelles règles si elles ne sont pas déjà en vigueur et désapplique les anciennes si elles sont bien en vigueur. C'est en cela que sera utile son **set<Rule> rules_history**. Appliquer une règle sera tout simplement ajouter la propriété concernée au label concerné dans la map **properties** et la désappliquer sera retirer la propriété concernée au label concerné dans la map.

5 Gestion de mémoire

Le focus porté par le cours sur la gestion efficace de la mémoire s'est traduit par l'attention que nous y avons portée pour notre projet. Elle s'est faite en plusieurs points.

Utilisation à bon escient de la STL. Les conteneurs utilisés ne sont pas choisis aléatoirement mais en fonction de leur utilité.

Tout ce qui est créé doit être détruit. La totalité de la mémoire que nous allouons est libérée. **Game** lors de sa destruction détruit les **Character**, **Action**, **Board** et **Ruler** dont il est le propriétaire. Conformément à la *Rule of Three*, il devrait redéfinir le constructeur de copie et l'opérateur **=**. Nous avons cependant supprimé ces fonctions, notre modélisation rendant leur utilisation non judicieuse.

Ne copier que ce qui doit être copié. Une bonne utilisation des références et de **const** nous a permis de limiter les copies et modifications non autorisées de nos objets. **Position** (qui est un objet léger tout de même) a sciemment parfois été copié car souvent utilisée par des fonctions de calcul qui la modifient.

6 Les plus réalisés

Notre travail nous a porté au delà des objets et règles les plus fréquents du jeu. Nous avons ainsi traité l'action de **PULL** évoquée plus haut qui permet au joueur de tirer à sa suite les objets dont les propriétés le permettent. Nous avons ajoutés les objets **KEY** et **DOOR** et les propriétés les concernant (**SHUT** et **OPEN**). Nous avons aussi pensé aux règles de la forme **SUBJECT IS SUBJECT**.

WALL IS BABA fera attribuer à WALL toutes les propriétés de BABA. Pour aller encore plus loin, nous avons g  rer les tautologies. Le fait qu'un objet ait    la fois les propri  t  s YOU et DEFEAT est synonyme de d  faite. De m  me YOU + WIN donne une victoire automatique. Ajout      tout cela, nous avons un environnement du jeu soign   avec une interface graphique agr  able et une suite de niveaux   ditables et extensibles.

7 Notes

Les sites kbhgames-babaisyou et babaiswiki nous ont servi de r  f  rences pour comprendre les r  gles du jeu m  me si certaines d  cisions ont   t   arbitraires de notre part.

8 Conclusion

BABA IS YOU est finalement un jeu tr  s int  ressant du fait sa singularit   quant    la gestion de ses r  gles. Concevoir un tel jeu a donn   lieu    nombreuses r  flexions et changements de mod  lisation. Cela nous a permis de mettre en pratique les notions de gestion rigoureuse de la m  moire et de conception logicielle qui font loi lorsqu'on est confront   au C++. Le r  sultat a   t   un travail fructif qui reste tout de m  me extensible. Le traitement du AND, le jeu sur les touches de contr  le ou encore l'int  gration des objets et propri  t  s mis de c  t   sont parmi d'autres des id  es d'am  lioration...